

Article

基于GPT的空间数据库文本到SQL转换

王辉^{1,*†}, 郭力^{2†}, 梁宇彬¹, 刘乐³ 和 黄家晋^{4,5}

¹ 天津师范大学地理系, 中国天津 300387; ybliang@tjnu.edu.cn ² 信息工程大学地理空间信息研究所, 中国郑州 450001; gl_750312@163.com ³ 天津大学建筑工程学院, 中国天津 300350; liule0328@tju.edu.cn ⁴ 北京工业大学信息技术学院, 中国北京 100124; jhuang@bjut.edu.cn ⁵ 北京脑信息与智慧服务国际合作基地, 中国北京 100124
* 通信作者: wanghui@tjnu.edu.cn [†] 这些作者对本工作贡献相同。

摘要

面向空间数据库的文本到SQL (Text-to-SQL) 技术能够将自然语言问题转换为相应的SQL查询, 使非专业人士也能轻松访问空间数据, 这一技术已经引起研究人员越来越多的关注。之前的研究主要集中在基于规则的方法。然而, 这些方法在处理复杂或未知的自然语言问题时存在局限性。虽然可以训练先进的机器学习模型, 但它们通常需要大量标注的训练数据, 而空间数据库方面的这类数据严重不足。近年来, 生成式预训练变换器 (GPT) 模型作为关系型数据库Text-to-SQL任务的有希望的范式出现, 其性能依赖于精心设计的提示 (prompts)。针对空间数据库数据集严重缺乏的问题, 我们创建了一个公开可用的数据集, 支持英文和中文。此外, 我们提出了一种基于GPT的方法来为空间数据库构建提示 (prompts), 该方法结合了



学术编辑: 沃尔夫冈·凯恩茨 和 埃利塞奥·克莱门蒂尼

收到: 2025年5月20日
修订: 2025年7月18日
接受: 2025年7月22日
发布: 2025年7月24日

引用: Wang, H.; Guo, L.; Liang, Y.; Liu, L.; Huang, J. 基于 GPT 的空间数据库文本到 SQL 转换。
ISPRS Int. J. Geo-Inf. **2025**, *14*, 288. <https://doi.org/10.3390/ijgi14080288>

版权: © 2025 作者所有。由 MDPI 代表国际摄影测量与遥感学会出版。许可方 MDPI, 瑞士巴塞尔。本文为开放获取文章, 根据知识共享署名 (CC BY) 许可协议 (<https://creativecommons.org/licenses/by/4.0/>) 的条款和条件分发。

关键词: 空间数据库; 文本到SQL; GPT

1. 引言

根据[1]的一项研究, 80%的大数据是空间数据。因此, 空间数据库因提供高度专业化的空间查询和分析能力, 已成为管理这些数据的核心工具。虽然专家可以使用SQL与这些数据库进行交互, 但对非专家来说, 这种语言可能具有挑战性, 从而限制了他们充分利用空间数据库管理的数据的能力。

用于空间数据库的文本到SQL (Text-to-SQL) 通过将自然语言问题/文本 (NLQ/Text) 翻译为对应的SQL查询, 如图1所示, 提供了一种有前景的解决方案。这使非专业人员能够轻松访问空间数据, 并且越来越受到研究人员的关注。

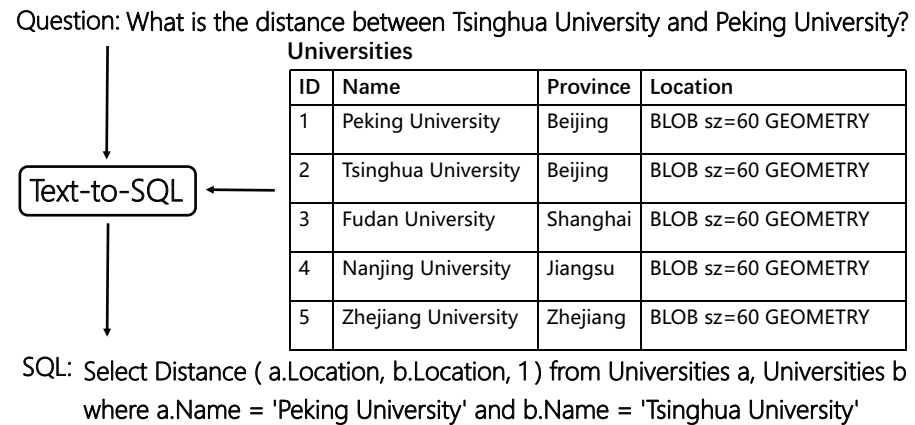


图1. 空间数据库文本到SQL的示例。

以往的研究已经调查了将自然语言问题翻译为空间数据库的 SQL 查询的问题 [2–4]。这些研究可以总结为基于规则的方法。虽然这些方法不需要大量的训练数据，但在处理复杂或未知的自然语言问题时存在局限性。

这些限制理论上可以通过训练一个机器学习模型来解决。然而，该模型需要一个标注的训练数据集，该数据集不仅涵盖各种空间自然语言问题，还包括对应的正确 SQL 查询。此外，数据集必须足够大，以确保训练出的机器学习模型能够达到令人满意的性能。不幸的是，由于空间查询的复杂性，创建这样的数据集既耗时又费力。

最近，生成式预训练变换器（GPT）模型，如 GPT-4 和 DeepSeek，在广泛的任务中表现出了令人印象深刻的性能。这些模型旨在接受一种基于文本的输入，即提示，并生成类似人类的响应。在使用 GPT 模型生成准确的 SQL 查询时，一个关键因素是提示的构建。提示应基于自然语言问题及相应的数据库模式。研究 [5–7] 表明，在提示中包含与自然语言问题相关的示例，可以使 GPT 模型生成的输出更符合用户的期望。

在本文中，我们提出了一种基于GPT的方法来构建空间数据库的提示，该方法将地理和空间数据库知识融入提示中，并且只需少量训练样本，例如1、3或5个示例。此外，我们创建了一个包含空间自然语言问题及其对应SQL查询的数据集，该数据集提供英文和中文版本。该数据集可用于模型训练和性能评估。此外，我们的框架只需少量训练样本，例如1、3或5个示例，就能有效引导GPT模型生成准确的SQL查询。

本文其余部分的组织结构如下。第2节回顾了空间数据库Text-to-SQL任务的相关工作。第3节介绍了我们提出的方法。第4节概述了用于评估我们方法的实验设计。第5节展示了实验结果。第6节讨论了我们方法的局限性。最后，第7节对本文进行总结。

2. 相关工作

高质量的数据集对于研究和评估文本到SQL任务至关重要。研究人员已经开发了几个用于文本到SQL的标注数据集，包括GeoQuery [8]、Restaurants [9]、Scholar [10]、WikiSQL [11]、Spider [12]、CSpider [13]、DuSQL [14]

Spider-DK [15]、Spider-Syn [16] 和 Spider-SSP [17]。这些数据集由 <NLQ/文本、SQL> 对组成，并广泛用于文本到 SQL 的学术研究。

为了促进文本到SQL（Text-to-SQL）技术从学术研究向实际应用的转移，阿里巴巴集团与香港大学等机构合作，推出了BIRD数据集 [18]。该数据集专注于广泛且真实的数据库内容，作为数据库的大规模基准测试数据。

上述提到的这些数据集都专注于关系型数据库。尽管GeoQuery和Restaurants数据集与地理领域相关，但它们仍然缺乏用于空间查询的相关数据。因此，专门为空间数据库设计的文本到SQL数据集仍然存在显著的空白。

深度学习的发展，以及大规模文本到SQL数据集的可用性，显著加快了关系数据库文本到SQL的研究。已经提出了几种方法，包括Seq2Seq [19]、IRNet [20]、RYANSQL [21]和SQLova [22]，以及预训练模型如TaBERT [23]、TaPas [24]和Grappa [25]，并且这些方法已经展示了有希望的结果。

在这些方法中，SpatialNLI [26,27] 主要关注解决空间实体的语义歧义，例如确定自然语言问题中的‘Mississippi’指的是河流还是州。然而，SpatialNLI 使用的训练数据集仍然是为关系数据库而设计的，而不是空间数据库。

由于现有 Text-to-SQL 数据集中缺乏空间查询的数据，使用当前 Text-to-SQL 数据集通过深度学习方法训练的模型在直接应用于空间数据库的 Text-to-SQL 任务时通常会面临挑战。因此，目前针对空间数据库的 Text-to-SQL 研究主要采用基于规则的方法 [2–4]。这在很大程度上是因为基于规则的方法不需要大规模的训练数据集。然而，虽然基于规则的方法适用于简单的用户查询，但对于各种场景或领域，提前设计 SQL 模板是具有挑战性的，甚至是不可能的。

最近，GPT 模型已成为 Text-to-SQL 任务的新范式 [28–30]。与以往的研究不同，基于 GPT 的 Text-to-SQL 解决方案的核心挑战在于如何引导模型生成正确的 SQL 查询，具体来说就是如何构建有效的提示。许多研究已经提出了各种提示构建方法 [4,7,31–33]。然而，这些方法主要集中于关系型数据库，并没有考虑空间数据库的独特复杂性。因此，将它们直接应用于空间数据库的 Text-to-SQL 任务并不能获得令人满意的结果，正如我们的实验将展示的那样。

地球科学中对时空数据和建模的需求日益增长，这极大地推动了地理空间代码生成技术的发展。参考文献 [34–38] 专门研究了使用大型语言模型生成地理空间分析代码的情况。对于地理空间SQL生成，参考文献 [39] 简要报告了使用GPT模型进行面向空间数据库的文本到SQL任务的潜力。

尽管取得了这些进展，用于构建针对空间数据库的 Text-to-SQL 任务提示的方法仍然有限。在本研究中，我们旨在开发一种专门针对空间数据库的 Text-to-SQL 任务的提示构建方法，同时创建一个专门用于训练和评估该任务的数据集。

3. 方法

3.1. Overall Framework

我们的方法包括两个步骤：(1) 我们将提示发送给GPT模型，该模型根据提供的提示生成SQL查询。(2) 生成的SQL查询随后在空间数据库管理系统（SDBMS）中执行。如果没有发生异常，则该SQL查询被视为GPT模型的最后预测。如果检测到异常，则从异常反馈中提取一个模式提示（例如，“数据库中不存在表‘t’，不应使用该表”），并将其重新整合到提示中，然后重复该过程。一旦达到预定义的重复杂数阈值，程序即终止。

我们方法中的提示包含六个部分：(1) SDBMS 知识，(2) 与目标问题相似的自然语言问题及其对应的 SQL 查询示例，(3) 数据库模式，(4) 数据库中每个表的地理描述，(5) 模式提示，(6) 目标问题。

图2展示了我们方法的整体框架，而图3提供了提示的一个片段。提示的六个组成部分的详细信息将在以下小节中说明。

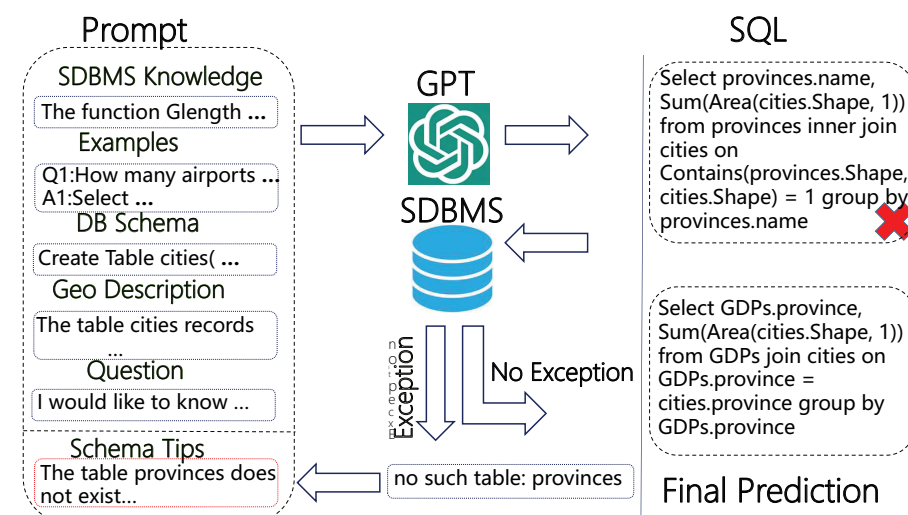


图2. 我们方法的整体框架。

3.2. SDBMS Knowledge

SDBMS 通常通过关系数据库的空间扩展来实现，并且提供多种类型，例如带有 PostGIS 的 PostgreSQL、Oracle Spatial、SQL Server Spatial 以及带有 SpatiaLite 的 SQLite。这些 SDBMS 提供空间函数以支持各种空间查询和分析能力。然而，具有相同功能的空间函数名称在不同的 SDBMS 中可能不同。例如，在计算几何对象的面积时，该函数在 SQL Server 中被命名为 STArea，而在 SpatiaLite 中，对应的函数仅被称为 Area。此外，在单个 SDBMS 中，这些函数的名称在不同版本之间可能会有所不同。

这些不一致可能会导致 GPT 模型预测的 SQL 查询中的函数名称与所使用的 SDBMS 提供的函数名称不匹配。

此外，在同一个 SDBMS 中，用于计算的一些空间函数可能具有不同的参数设置，这可能导致结果不同。例如，在 SpatiaLite 中，Distance (geom1 Geometry, geom2 Geometry, 1) 和 Distance (geom1 Geometry, geom2 Geometry) 的结果不同，前者提供了更精确的结果。

You are an expert in the SpatiaLite spatial database and highly skilled in writing SQL queries.

In SpatiaLite,

The function Area(s Surface, 1) returns the area of s, measured in meters.

The function GLength(c Curve, 1) returns the length of c, measured in meters.

...

/* Some SQL examples are provided based on similar problems: */

/* Answer the following: I would like to know the total area of the Beijing-Tianjin-Hebei region. Select Sum(Area(provinces.Shape, 1)) from provinces where provinces.name = '北京市' or provinces.name = '天津市' or provinces.name = '河北省'

/* Answer the following: Which provinces border Guangdong Province, and what is the area of province? */

Select b.name , Area(b.Shape, 1) from provinces a inner join provinces b On Touches(a.Shape, b.Shape) 1 where a.name = '广东省'

...

/* Given the following database schema: */

```
CREATE TABLE 'airports' (
  IATA TEXT PRIMARY KEY,
  runway_number INTEGER,
  airfield_area_class TEXT,
  Name TEXT,
  Location POINT)
```

The 'airports' table records information about various airports in China.

```
CREATE TABLE 'cities' (
  name TEXT PRIMARY KEY,
  administrative_division_code TEXT,
  province TEXT REFERENCES "GDPS" (province),
  Shape MULTIPOLYGON)
```

The 'cities' table records information about various cities in China.

/* Answer the following with no explanation: What is the total area of the three northeastern provinces? The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang. Liaoning is represented name '辽宁省', Jilin is represented by the name '吉林省', and Heilongjiang is represented by the name '黑龙江省'. The table "provinces" does not exist in the database and should not be used. */

```
SELECT
```

图 3. 我们方法中提示的一个片段。

为了确保 GPT 模型能够准确使用所使用 SDBMS 提供的功能并满足用户的特定计算需求，我们首先在提示中指定所使用的 SDBMS。例如，“你是 SpatiaLite 空间数据库的专家，并且非常擅长编写 SQL 查询”。在我们的实验中，我们使用 SpatiaLite。

此外，我们提供了对提示中一些常用函数的清晰解释，例如 GLength、Area、Distance、Centroid、Touches 和 Intersects。例如，“函数 GLength(c Curve, 1) 返回曲线 c 的长度，单位为米。”

3.3. Examples

我们使用掩码问题相似性选择方法 [6,33] 来衡量目标问题与训练数据集中问题之间的相似性。接下来，我们从训练数据集中选择 K 个最相似的问题及其对应的 SQL 查询作为示例。

3.4. Database Schema

数据库模式定义了数据库的结构，包括表、列、表之间的关系、数据类型和其他元素。在本节中，我们使用代码表示法 [40,41] 来描述相应的数据库模式。具体来说，如图3所示，它直接呈现了“CREATE TABLE” SQL 语句。

3.5. Geographic Descriptions

在 Text-to-SQL 任务中，在提示中为每个表添加描述可能有助于澄清表的语义并减少歧义。在本节中，我们在提示中为每个表添加地理区域描述。这是基于我们的观察：在空间数据库中，每个表中的数据通常对应于特定的地理区域。例如，在我们使用的交通数据库中，‘roads’ 表包含南京的道路信息。有一类问题，例如“南京每条道路的总长度是多少？”和“每条道路的总长度是多少？”，我们称之为与区域相关的问题。这两个问题在本质上是等价的。然而，当面临“南京每条道路的总长度是多少？”这个问题时，GPT 模型经常会将‘南京’误解为数据库中的一个列值，从而生成错误的 SQL 查询。

为了解决这个问题，我们在提示中明确描述与每个表相关的地理区域。例如，对于“roads”表，我们包括描述“‘roads’表记录南京的道路信息。”

3.6. Question

在本节中，我们采用 DAIL [33] 中的简单问题表示方法，其中自然语言问题前缀为“Answer the following with no explanation:”，后跟“SELECT”。在生成 SQL 查询时，GPT 模型有时会对某些术语提供解释。短语“with no explanation”有助于确保 GPT 仅返回答案。

3.7. Schema Tips

在文本到SQL的任务中，利用执行反馈进行自我纠正可能有助于提高预测准确性。在本节中，我们重点处理三种类型的反馈：不存在的表名、不存在的列名以及模糊的列名。这是因为基础数据（例如行政区划或道路）经常出现在多个空间数据库中。因此，示例中的表或列名称可能与目标问题中的名称或含义相同或相似。如图3所示，术语“provinces”同时出现在示例和目标问题中。这种重叠可能导致GPT模型生成引用示例中表或列的SQL查询，而这些表或列在给定的数据库模式中并不存在。

为了使 GPT 模型能够根据给定的数据库 schema 正确引用表或列，我们从步骤 (2) 的异常反馈中提取三种类型的 schema 提示，并将它们整合到提示中。

- 当异常反馈为“no such table: t”时，我们提取表名‘t’，并在提示中附加以下句子：“数据库中不存在表‘t’，不应使用该表。”
- 当异常反馈为“no such column: c”时，我们提取列名‘c’及其对应的表名‘t’，并将以下句子附加到提示中：“列‘c’在表‘t’中不存在，不应使用。”
- 当异常反馈为“模糊的列名: c”时，我们在提示中追加以下句子：“多个表中的列可能有相同的名称，因此应避免歧义。”

4. 实验

4.1. Experiment Dataset

由于用于文本到 SQL 任务的空间数据库数据集有限，我们开发了一个专门的数据集来评估我们提出的方法。用于关系数据库的数据集，如 BIRD [18] 和 Spider [12]，是最常用的数据集之一，通常使用世界上最流行的数据库引擎 SQLite 构建。为了保持一致性，我们的数据集建立在 SpatiaLite 之上，SpatiaLite 扩展了 SQLite 的功能，提供全面的空间 SQL 能力。

该开发的数据集包含四种类型的空间数据库：Ada（行政区划）、Edu（教育）、旅游和交通。所有数据均来源于中国的真实信息。

在一些用于关系数据库的数据集中，例如 BIRD [18]，一个 Text-to-SQL 对包括文本和 SQL 查询。文本不仅包含问题，还包含证据，证据指的是通常由专家提供的补充知识。为了确保我们的数据集支持中文和英文问题，我们在每个 Text-to-SQL 对中添加了一个“name”字段，该字段指定数据库中某些短语的实际值。

例如，在问题“东北三省的总面积是多少？东北三省指辽宁省、吉林省和黑龙江省。”中，短语‘Liaoning’在数据库中对应对应‘辽宁省’，‘Jilin’对应‘吉林省’，‘Heilongjiang’对应‘黑龙江省’。因此，在‘name’字段中，我们包括描述“辽宁由名称‘辽宁省’表示，吉林由名称‘吉林省’表示，黑龙江由名称‘黑龙江省’表示”。

此外，我们参考了 CSpider [13] 的方法，这是从英文数据集 Spider [12] 翻译而来的中文数据集，用于为问题、证据和名称的字段提供中文对应。具体细节列在表 1 中。

在空间数据库中，某些自然语言问题可以使用包含非空间列或空间列的 SQL 来执行。例如，Ada 数据库中的“rivers”表包含九列，包括一个非空间列“length”和一个空间列“shape”，后者是“geometry”类型。要查询一条河流的长度，可以直接访问“length”列或使用空间函数“GLength(shape, 1)”来计算。类似地，Edu 数据库中的“cities”表包含十列，包括一个非空间列“province”和一个空间列“shape”，而 Edu 数据库中的“provinces”表包含四列，包括一个空间列“shape”。要确定一个城市所在的省份，可以查询“cities”表中的“province”列，或者使用

空间函数 ‘Within (cities.shape, provinces.shape)’ 用于确定城市是否位于特定省份内。

表 1. 我们开发的数据集中一个文本到 SQL 的示例，其中字段 ‘questionCHI’、‘evidenceCHI’ 和 ‘nameCHI’ 分别表示字段 ‘question’、‘evidence’ 和 ‘name’ 的中文对应项。

question: What is the total area of the three northeastern provinces?
questionCHI: 东三省的总面积是多少?
evidence: The three northeastern provinces refer to Liaoning, Jilin, and Heilongjiang.
evidenceCHI: 东三省指辽宁省, 吉林省, 黑龙江省.
name: Liaoning is represented by the name ‘辽宁省’, Jilin is represented by the name ‘吉林省’, and Heilongjiang is represented by the name ‘黑龙江省’.
nameCHI: 辽宁省以‘辽宁省’为名称表示, 吉林省以‘吉林省’为名称表示, 黑龙江省以‘黑龙江省’为名称表示.
SQL: Select Sum (Area (Shape, 1)) from cities where province in (‘辽宁省’, ‘吉林省’, ‘黑龙江省’)

在实际应用中，我们可以计算 ‘SQL’ 字段中的短语与数据库中相应列值之间的相似度，选择相似度最高的值作为 ‘SQL’ 字段中的实际值。

我们将这些非空间列称为派生列，例如“长度”和“省份”，其值可以从空间列或函数中推导或计算得出。为了增强我们的数据集并全面评估我们方法的性能，我们对Ada、Edu和Traffic数据库进行转换，在一个版本中保留派生列，在另一个版本中删除它们。这个过程产生了每个数据库的两个版本：含派生列的Ada¹、Edu¹、Traffic¹ ()，以及不含派生列的Ada²、Edu²、Traffic² ()。

我们参考了文献[2,3]中的空间问题（每篇包含100个问题，且空间数据库及相应的SQL查询未公开），以及BIRD和Spider中的一般问题，设计了一套200个问题。在每种数据库类型中（Ada¹和Ada²，Edu¹和Edu²，Traffic¹和Traffic²），问题是相同的。在Ada¹、Edu¹和Traffic¹数据库中，由于存在派生列，空间问题可以通过多条SQL查询解决，既可以使用非空间列（一般SQL查询），也可以使用空间列（空间SQL查询）。相比之下，在Ada²、Edu²和Traffic²数据库中，由于派生列已被移除，空间问题需要使用空间列（即空间SQL查询）。为了增强我们的数据集，我们尽可能为每个200个问题提供多条SQL查询，总共产生了296条独特的SQL查询。开发的数据集的详细信息

表 2. 我们开发的数据集的详细统计信息。

Name	Tables	Columns	PKs	FKs	Qs	Rs	SQLs
Ada ¹	6	53	6	1	56	7	90
Ada ²	6	49	6	0	56	7	56
Edu ¹	3	26	3	3	44	0	93
Edu ²	3	22	3	0	44	0	44
Tourism	6	57	6	2	33	1	33
Traffic ¹	11	55	11	4	67	26	81
Traffic ²	11	51	11	3	67	26	67

在表格中，横轴表示每个数据库中的表格数量、列数、主键 (PK)、外键 (FK)、问题 (Qs)、与地理区域相关的问题 (Rs，例如第3.5节中的“南京每条道路的总长度是多少？”)，以及SQL查询数量。

4.2. Experiment Methods

我们使用基于 GPT 模型的以下方法进行实验评估：

- **SSpa**：我们将我们的方法命名为SSpa，它由六个组成部分构成。其中，示例、数据库模式和问题是基于GPT的关系数据库Text-to-SQL任务方法的常见组成部分，而SDBMS知识、地理描述和模式提示则代表了我们的创新方面。
- **DAIL-SQL**：DAIL-SQL 由阿里巴巴集团开发，是一种高效且有效的方法，旨在优化 GPT 模型在 Text-to-SQL 任务中的应用。值得注意的是，配备 GPT-4 的 DAIL-SQL 在 Spider 排行榜上以 86.6% 的执行准确率获得第一名，创下了新的基准。DAIL-SQL 是基于 GPT 的关系数据库 Text-to-SQL 方法的优秀代表，主要由示例、数据库模式和问题组成。因此，我们将 DAIL-SQL 作为我们的基线方法。
- **SSpa-SDBMS**：与SSpa方法相比，SSpa-SDBMS去除了SDBMS知识组件，同时保留地理描述和模式提示。
- **SSpa-Geo**：与SSpa方法相比，SSpa-Geo移除了地理描述组件，同时保留了SDBMS知识和模式提示。
- **SSpa-Tips**：与SSpa方法相比，SSpa-Tips移除了模式提示组件，同时保留了SDBMS知识和地理描述。

为了确保公平的比较，鉴于 DAIL-SQL 在 GPT-4 上取得最佳性能，我们在本研究中评估的所有方法都使用 GPT-4。为了增强我们方法的稳健性和可靠性，我们使用 DeepSeek 进行了补充对比实验。所有方法均通过 OpenAI API 访问。

为了消除任何来自随机性的影响，我们默认将温度参数设置为0，适用于GPT-4和DeepSeek。此外，第（2）步中重复的最大次数限制为五次。无论最后预测的SQL查询是否遇到执行异常，它都被视为最终预测。

4.3. Evaluation Metrics

在 Text-to-SQL 任务中最常用的评估指标是精确匹配（EM）和执行准确性（EX）。对于 EM，评估是基于预测的 SQL 查询的每个部分是否与真实的 SQL 查询完全匹配。只有当查询的所有部分完全匹配时，预测才被认为是正确的。对于 EX，它通过比较预测 SQL 查询和真实 SQL 查询在同一数据库上的执行结果来评估预测。只有当执行结果完全相同时，预测才被认为是正确的。

在空间数据库中，派生列的存在带来了一个挑战，即对于一个给定的问题可能存在多个有效的 SQL 查询。此外，GPT 模型在编写 SQL 查询时通常会表现出自己独特的风格，这可能导致对于相同的问题生成不同的 SQL 查询。因此，在评估空间数据库在 Text-to-SQL 任务中的方法时，使用 EM 指标可能存在偏差，因为它要求 SQL 语法完全匹配。相比之下，使用 EX 指标直接比较执行结果，更有意义且更实用。因此，在我们的实验中，我们仅使用 EX 作为评估指标。

EX 被定义为一个百分比，可以表示为

$$EX = \frac{|\text{Correctly predicted}|}{|\text{All predicted}|} \times 100\%$$

5. 结果

我们基于开发的空数据数据集进行了两组实验。在第一组实验中，我们通过结合Ada¹、Edu¹、Tourism和Traffic¹数据库及其各自样本创建了数据集1。最初，我们在Ada¹和Edu¹ (的100个问题及其对应的SQL查询)上进行训练，并在Tourism和Traffic¹的100个问题及其对应的真实SQL查询)上进行测试，应用所有预测方法。然后我们交换角色，在Tourism和Traffic¹ (的100个问题及其对应的SQL查询)上进行训练，并在Ada¹和Edu¹ (的100个问题及其对应的真实SQL查询)上进行测试。最后，我们使用EX评估数据集1中所有200个预测。在此上下文中，训练指的是从训练数据集中选择与目标问题最相似的前K个问题及其对应的SQL查询作为提示中的示例。

第二组实验遵循相同的程序，但使用Ada²、Edu²、Tourism和Traffic²来形成数据集2。具体来说，我们交替进行：在Ada²和Edu² (上训练，在Tourism和Traffic²)上测试；以及在Tourism和Traffic² (上训练，在Ada²和Edu²)上测试，再次评估数据集2中的所有200个预测。

虽然数据集1和数据集2中的问题是相同的，但在数据集2中删除派生列和相关外键导致Ada、Edu和Traffic数据库的架构发生了显著变化。

5.1. Performance Evaluation of Each Method

对于所有方法，我们在数据集1和数据集2上对GPT-4和DeepSeek进行了实验，测试的场景包括提示中没有示例的情况，以及包含1个、3个或5个示例的情况。我们将提示中没有示例的场景称为零样本（0-shot），而包含1个、3个和5个示例的场景分别标记为一样本（1-shot）、三样本（3-shot）和五样本（5-shot）。这些实验的结果总结在表3和表4中。

表3. 各方法在GPT-4上的性能 (%)。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	44.5	52.5	59.5	65.5	34.0	47.0	59.0	57.5
SSpa	85.0	83.5	85.0	87.5	78.0	80.0	81.0	79.0
SSpa-SDBMS	50.0	56.0	61.5	70.5	38.0	48.0	65.5	73.0
SSpa-Geo	80.5	77.5	78.0	77.5	75.0	71.0	71.5	68.5
SSpa-Tips	79.0	80.0	83.5	82.5	74.5	77.5	78.0	75.5

表 4. 各方法在 DeepSeek 下的性能 (%)

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	50.5	63	65.5	70.5	38.0	57.5	62.5	65.5
SSpa	82.5	83	82.5	86	75.5	81	81	81
SSpa-SDBMS	51.5	65	72	78.5	41	65.5	76.5	77.5
SSpa-Geo	79.5	79	79	79	72	76	74.5	73.5
SSpa-Tips	80.5	81	80.5	83.5	68.5	77	78.5	76.5

首先，我们比较方法SSpa、SSpa-SDBMS、SSpa-Geo、SSpa-Tips和DAIL-SQL之间的性能差异。如表3和表4所示，无论使用GPT-4还是DeepSeek，方法SSpa、SSpa-SDBMS、SSpa-Geo和SSpa-Tips在数据集1和数据集2上始终优于基线方法DAIL-SQL。具体来说，使用GPT-4的SSpa方法在数据集1上取得了最高的87.5%性能。

在5-shot场景中以及在3-shot场景中，Dataset 2上达到81%，在两个场景中都比DAIL-SQL高出22%。类似地，使用DeepSeek的SSpa方法在5-shot场景中Dataset 1上达到最高性能86%，在Dataset 2上达到81%，在两个场景中至少比DAIL-SQL高出15.5%。这些性能差异符合我们的预期，因为DAIL-SQL是专门为关系型数据库设计的，而空间数据库具有其独特的特性。SSpa方法及其变体（SSpa-SDBMS、SSpa-Geo和SSpa-Tips）通过将所使用的空间数据库管理、地理描述或模式提示的特定知识整合到模型的提示中，提高了性能，从而提升了其在空间数据库Text-to-SQL任务中的有效性。

接下来，我们分析 SSpa 和 DAIL-SQL 方法在数据集 1 和数据集 2 上的性能差异。如表 3 和表 4 所示，无论是使用 GPT-4 还是 DeepSeek，这两种方法（SSpa 和 DAIL-SQL）在所有场景（0-shot、1-shot、3-shot 和 5-shot）中，在数据集 1 上的表现始终优于数据集 2。该结果可归因于数据集 1 和数据集 2 之间 SQL 查询的差异。在数据集 1 中，由于存在派生列，问题可以通过通用 SQL 查询和空间 SQL 查询进行解决。然而，在数据集 2 中，由于移除了派生列，问题只能通过空间 SQL 查询来回答，而空间 SQL 查询通常比通用 SQL 查询更具挑战性。此外，与关系数据库相比，空间数据库提供的可用知识较少，这导致 GPT 模型在训练中利用的空间数据库知识减少，从而可能导致生成空间 SQL 查询的性能较差。

最后，我们评估了 SSpa 方法在各种场景下（0-shot、1-shot、3-shot 和 5-shot）的性能。如图 4 所示，对于 GPT-4 和 DeepSeek，SSpa 在非 0-shot 场景中均达到最佳性能，这表明在提示中加入相关示例可以提高 SQL 查询预测的准确性。然而，增加示例的数量并不总是能持续提高性能。例如，在 GPT-4 上，SSpa 在数据集 1 的 0-shot 中优于 1-shot，在数据集 2 的 3-shot 中优于 5-shot。前者可能是由于选择的单个示例不合适或不具代表性，而后者可能是 5-shot 场景中示例间存在干扰。相比之下，DeepSeek 并未出现这些异常，这可能是由于模型架构和训练的差异。不管怎样，GPT-4 和 DeepSeek 在非 0-shot 场景中都取得了最佳结果。

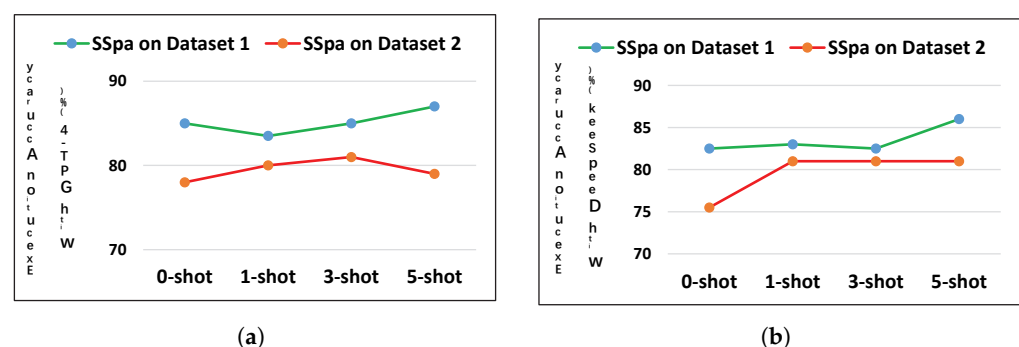


图 4. 一张折线图清晰地展示了 SSpa 在数据集 1 和数据集 2 上的 0-shot、1-shot、3-shot 和 5-shot 的表现。(a) SSpa 与 GPT-4 的表现。(b) SSpa 与 DeepSeek 的表现。

5.2. Effect of Each Component in the Prompt

如表3和表4所示，我们的方法SSpa在所有场景（0-shot、1-shot、3-shot和5-shot）下，无论使用GPT-4还是DeepSeek，都优于SSpa-Tips、SSpa-SDBMS和SSpa-Geo。这表明在我们的方法中移除提示的任何组件都会对其性能产生负面影响。

首先，我们研究模式提示组件在 SSpa、SSpa-SDBMS 和 SSpa-Geo 方法提示中的作用。在我们的框架中，如果在执行 GPT 生成的 SQL 查询时发生异常，我们会从异常反馈中提取模式提示。提取的模式提示随后被纳入原始提示，并将增强后的提示重新提交给 GPT 以进行新的预测。因此，我们分析在包含模式提示后纠正的最初失败查询的数量。例如，对于问题“东北三省的总面积是多少？”的提示被发送给 GPT，GPT 生成的 SQL 查询为“Select Sum (Shape_Area) from cities where province in (‘辽宁省’ , ‘吉林省’ , ‘黑龙江省’) ”。执行时，SQL 查询产生错误“no such column: Shape_Area”。我们提取模式提示“表 ‘cities’ 中不存在列 ‘Shape_Area’，不应使用”，将其纳入原始提示，并重新发送给 G

我们观察到，在1-shot场景下，使用GPT-4的SSpa-SDBMS在数据集1上的SQL查询正确预测没有提升，而在其他所有场景中，加入schema tips组件都提高了预测准确率。值得注意的是，当在提示中加入schema tips组件时，我们的SSpa方法在数据集1和数据集2上最多可以准确预测八条SQL查询，占总共200条预测的4%。这表明，在面向空间数据库的Text-to-SQL任务中，当GPT预测的SQL查询遇到模式错误（如缺少表或列、列名模糊）时，提取schema tips并将其融入提示中可以提高GPT的SQL查询预测准确率。

表5. 使用GPT-4时，通过每种方法添加模式提示组件纠正的最初失败SQL查询预测的数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
SSpa	1	2	2	8	1	3	7	8
SSpa-Tips	-	-	-	-	-	-	-	-
SSpa-SDBMS	2	0	3	3	1	3	4	4
SSpa-Geo	2	2	2	7	2	3	4	6

表 6. 使用 DeepSeek 时，每种方法通过添加模式提示组件纠正的最初失败的 SQL 查询预测数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
SSpa	1	2	2	3	1	1	7	8
SSpa-Tips	-	-	-	-	-	-	-	-
SSpa-SDBMS	1	3	7	5	1	5	8	9
SSpa-Geo	2	4	3	5	1	2	7	12

接下来，我们将研究地理描述组件的影响。地理描述组件是指在SSpa方法的提示中为每个表格包含地理区域描述。如图3所示，‘airports’表格

被描述为“‘airports’表记录了中国各个机场的信息”。相比之下，SSpa-Geo 和 DAIL-SQL 方法的提示中并不包含这样的地理描述。如表 2 所示，200 个问题中有 34 个与地理区域相关。因此，我们评估每种方法在与地理区域相关的问题上正确 SQL 查询预测的数量。详细结果见表 7 和表 8。

我们观察到，在所有场景中，SSpa 在正确 SQL 查询预测数量上始终优于 DAIL-SQL 方法。唯一的例外是零样本场景，在该场景下，使用 GPT-4 的 SSpa 略逊于 SSpa-Geo（SSpa 正确预测了 31 个此类问题，而 SSpa-Geo 为 32 个），在其他所有场景中，SSpa 都超过了 SSpa-Geo。这表明，对于经常出现的特定区域问题，在提示中加入地理区域描述可以提高 GPT 模型的 SQL 查询预测准确性。

表7. 使用 GPT-4 时，各方法针对区域相关问题的正确 SQL 查询预测数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	9	8	10	11	6	7	11	11
SSpa	32	31	32	31	31	30	32	32
SSpa-Geo	27	16	13	13	32	17	12	11

表 8. 使用 DeepSeek 时各方法对区域相关问题的正确 SQL 查询预测数量。

Method	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
DAIL-SQL	10	16	15	16	6	18	17	18
SSpa	31	33	33	33	31	34	33	33
SSpa-SDBMS	13	21	23	24	10	23	31	31
SSpa-Geo	29	23	19	19	28	22	20	18
SSpa-Tips	30	31	31	31	27	32	31	30

最后，我们分析了SDBMS知识组件的作用。与SSpa-SDBMS方法相比，SSpa方法在其提示中加入了SDBMS知识，具体包括对某些函数的描述。因此，我们评估了通过引入SDBMS知识组件（即SSpa方法）成功纠正的、SSpa-SDBMS最初预测失败的SQL查询的数量，详细数据见表9。

我们观察到，在所有场景（零示例、单示例、三示例和五示例）中，大量最初失败的 SQL 查询预测在将 SDBMS 知识组件纳入提示后得到了成功纠正。在五示例场景中，这类成功纠正的数量至少为 19（占总共 200 个查询的 9.5%）对于 GPT-4，而 DeepSeek 为 14（7%）。这可以归因于对 SSpa-SDBMS 所使用的空间数据库缺乏了解。由于空间数据库管理系统的类型各异，每种类型具有不同的功能，SSpa-SDBMS 生成的 SQL 查询更可能调用与所使用的空间数据库管理系统不兼容的函数。例如，我们查看了在五示例场景中，针对问题“长江在湖北省的长度是多少？”由 SSpa-SDBMS 和 SSpa 方法使用 GPT-4 生成的提示。两个提示都未包含函数 GLength(c Curve, 1) 的示例。SSpa-SDBMS 的

涉及SDBMS知识，包括对GLength(c Curve, 1)函数的解释。对于这个问题，SSpa方法生成了正确的SQL查询“Select Sum(GLength(Intersection(rivers.shape, provinces.shape), 1)) AS LengthInHubei from rivers inner join provinces on Intersects(rivers.shape, provinces.shape) = 1 where rivers.name = ‘长江’ and provinces.name = ‘湖北省’”。相比之下，SSpa-SDBMS方法生成了一个错误的SQL查询“Select Sum(GLength(Intersection(rivers.shape, provinces.shape))) AS Length from rivers join provinces on Intersects(rivers.shape, provinces.shape) where rivers.name = ‘长江’ and provinces.name = ‘湖北省’”。该查询错误地使用了GLength(c Curve)函数而没有提供所需的参数1。值得注意的是，尽管在SSpa-SDBMS提示中没有关于GLength(c Curve)或GLength(c Curve, 1)函数的示例，该方法仍然生成了使用GLength(c Curve)的查询，尽管是错误的。这表明包括

表9. 通过SDBMS知识组件纠正的SSpa-SDBMS最初失败的SQL查询预测数量。

Model	Dataset 1				Dataset 2			
	0-Shot	1-Shot	3-Shot	5-Shot	0-Shot	1-Shot	3-Shot	5-Shot
GPT-4	73	57	49	36	86	66	35	19
DeepSeek	66	38	32	21	78	35	17	14

6. 限制

我们的工作也存在一些局限性。

首先，在提示的SDBMS知识组件中，我们仅手动指定了一些常用函数。如果提供更多与每个具体问题相关的函数，生成的SQL查询的准确性可能会得到提高。因此，在未来的工作中，我们将根据目标问题自动选择并在提示中包含适当的函数。

其次，在提示的模式提示组件中，我们只处理三种情况：不存在的表或列以及列名歧义。通过从异常反馈中提取更广泛的模式提示并将其纳入提示中，我们相信生成的 SQL 查询的准确性可以进一步提高。

最后，由于构建空间问题及其相应 SQL 查询的固有困难，我们的数据集仍然有限。因此，我们只识别出少数模式，例如为表添加地理区域描述。我们相信，随着数据集的增大和问题类型的更加多样化，可以发现更多更适合构建空间数据库提示的模式。当然，这三种局限性都代表了未来的研究方向，并且可以随着时间的推移逐步解决。

7. 结论

面向空间数据库的文本到 SQL 转换可以将自然语言问题翻译为基于给定数据库模式的相应 SQL 查询，从而使非专家更容易访问空间数据。这一能力已经引起了研究人员越来越多的关注。在我们的研究中，我们提出了一种基于 GPT 的方法，用于构建提示以解决空间数据库的文本到 SQL 任务。我们的方法引入了三项关键创新：空间数据库管理系统（SDBMS）知识的整合、地理描述以及数据库模式（schema）。

提示中的技巧。我们详细说明了方法论并呈现了系统实验。结果表明，包括SDBMS知识、地理描述和模式提示在内的所有三个组成部分，对于达到最佳性能都是必不可少的。省略任何一个组成部分都会对方法的准确性产生负面影响。此外，为进一步推进面向空间数据库的Text-to-SQL研究，我们创建了一个支持中英文的双语数据集。我们的工作有可能显著提高非专业人士使用空间数据库的便利性，促进其使用和应用。

作者贡献：概念构思，王辉和郭力；方法学，王辉和郭力；软件，刘乐和王辉；数据整理，刘乐；原稿撰写，王辉和郭力；审阅与编辑，王辉、郭力、梁宇斌和黄佳进；监督，王辉和郭力；资金获取，王辉。所有作者已阅读并同意已发表版本的手稿。

资金：本研究得到天津师范大学博士基金（项目编号52XB1619）及天津地理空间信息技术工程中心开放基金的资助。

数据可用性声明：支持本研究结果的数据可公开获取于 GitHub，链接为 <https://github.com/beta512/SpatialSQL>（访问日期：2025年7月14日）。

利益冲突：作者声明没有利益冲突

成立

参考文献

1. Leszczynski, A.; Crampton, J. 引言：空间大数据与日常生活。 *Big Data Soc.* 2016, 3, 1–6. [CrossRef]
2. Liu, M.; Wang, X.; Xu, J. NALSD：用于空间数据库的自然语言接口。发表于第18届国际空间与时间数据研讨会论文集，加拿大卡尔加里，2023年8月23–25日；第175–179页。
3. Liu, M.; Wang, X.; Xu, J.; Lu, H. NALSpatial：用于空间数据查询的有效自然语言转换框架。发表于第31届ACM地理信息系统国际会议论文集，德国汉堡，2023年11月13–16日；第4页。
4. Wang, X.; Liu, M.; Xu, J.; Lu, H. NALMO：针对移动对象数据库的自然语言查询转换。 *GeoInformatica* 2023, 27, 427–460. [CrossRef]
5. Liu, J.; Shen, D.; Zhang, Y.; Dolan, B.; Carin, L.; Chen, W. 什么样的示例适合作为GPT-3的上下文？发表于第3届深度学习架构知识抽取与整合研讨会论文集，都柏林，

13. Min, Q.; Shi, Y.; Zhang, Y. 一项针对中文 SQL 语义解析的初步研究。载于《2019 年自然语言处理实证方法会议暨第 9 届国际联合自然语言处理会议论文集》，香港，中国，2019 年 11 月 3–7 日；页 3652–3658。
14. Wang, L.; Zhang, A.; Wu, K.; Sun, K.; Li, Z.; Wu, H.; Zhang, M.; Wang, H. DuSQL: 大规模且实用的中文文本到 SQL 数据集。载于《2020 年自然语言处理实证方法会议论文集》，线上，2020 年 11 月 16–20 日；页 6923–6935。
15. Gan, Y.; Chen, X.; Huang, Q.; Purver, M.; Woodward, J.R.; Xie, J.; Huang, P. 提升文本到 SQL 模型对同义词替换的鲁棒性。载于《第 59 届计算语言学协会年会暨第 11 届国际联合自然语言处理会议论文集》，线上，2021 年 8 月 1–6 日；页 2505–2515。
16. Gan, Y.; Chen, X.; Purver, M. 探索尚未充分研究的局限性

34. 吴华; 沈志; 侯顺; 梁杰; 焦浩; 庆阳; 张翔; 李昕; 桂志; 关祥; 等。AutoGEEval: 一个用于GEE上地理空间代码生成的多模态自动化评估框架, 结合大型语言模型。 *ISPRS Int. J.-Geo-Inf.* 2025, 14, 256. [CrossRef]35. 侯顺; 焦浩; 沈志; 梁杰; 赵安; 张翔; 王健; 吴华。Chain-of-Programming (CoP): 赋能大型语言模型进行地理空间代码生成任务。 *Int. J. Digit. Earth* 2025, 18, 2509812. [CrossRef]36. 侯顺; 梁杰; 赵安; 吴华。GEE-OPs: 一个基于操作符的知识库, 用于在谷歌地球引擎平台上进行地理空间代码生成, 依托大型语言模型。 *Geo-Spat. Inf. Sci.* 2025, 1–22. [CrossRef]37. 侯顺; 沈志; 赵安; 梁杰; 桂志; 关祥; 李锐; 吴华。GeoCode-GPT: 一个用于地理空间代码生成的大型语言模型。 *Int. J. Appl. Earth Obs. Geoinf.* 2025, 138, 104456. [CrossRef]38. 侯顺; 赵安; 梁杰; 沈志; 吴华。Geo-FuB: 一种构建地理操作符-函数知识库的方法

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.